System Designing

04 May 2026 02:23 PM

LLD: (Low Level Design) It is basically detail design of the software how it is going to work at the code level.

- Actual classes and their relationships
- Method signatures and what they do
- Data structures to use
- Design patterns to apply
- Database schema details
- Algorithm implementations

OOPs:

History: Machine level Language - Binary
Assembly level language - MOV A,45H
Procedural - If else, functions etc
Object Oriented - **Class & Object** for real world

Objects has behaviour(functions) and characteristics
Eg: Class car{
String Brand;
String Model;
Boolean isEngineOn;
Start(){}
Stop(){}
Gearshift(){}
}

Pillars of OOPS:

- Abstraction - hides unnecessary details from client(who is interacting with it) show case on needed data.
  Eg:Car class has all those methods if drive() is calling the start,stop, drive() does not needs to know how start(),stop() are working.
  - One class has a abstarct method and properties and other class inheriets and override it.

```
Eg:
abstract class Animal
{
 // Abstract method (does not have a body)
 public abstract void animalSound();
 // Regular method
 public void sleep()
 {
  Console.WriteLine("Zzz");
 }
}
// Derived class (inherit from Animal)
class Pig : Animal
{
 public override void animalSound()
 {
    Console.WriteLine("The pig says: wee wee");
 }
}
class Program
{
 static void Main(string[] args)
 {
  Pig myPig = new Pig(); // Create a Pig object
  myPig.animalSound();  // Call the abstract method
  myPig.sleep();  // Call the regular method
 }
}
```

- Encapsulation : Combination of characteristics and behaviour where data needs to be secure.
  - Variables should not used by out side of the class where private access modifies should be used.
  - Restrict the change from outside.
  - If you want access or modify from outside the class use getter and setter.(Public methods of it from same class where private members are declared)
  - If you don't want to modify that's why you used encapsulation an bow you saying use getter and setter to manipulate the values the things is you can use validations before it in getter and setter. So data can manipulate logically validating all conditions.

    Eg:

```
using System;
class Car
{
    private string brand;
    private string model;
    private bool isEngineOn = false;
    private int currentSpeed = 0;
```

```
    private int currentGear = 0;
    private string tyreCompany;
    public Car(string brand, string model, string tyreCompany) {
        this.brand = brand;
        this.model = model;
        this.tyreCompany = tyreCompany;
    }
    public int getCurrentSpeed() {
        return currentSpeed;
    }
    public string GetTyreCompany()
    {
        return tyreCompany;
    }
    public void SetTyreCompany(string tyreCompany)
    {
        this.tyreCompany = tyreCompany;
    }
    public void StartEngine()
    {
        isEngineOn = true;
        Console.WriteLine($"{brand} {model} : Engine starts with a roar!");
    }
    public void ShiftGear(int gear)
    {
        currentGear = gear;
        Console.WriteLine($"{brand} {model} : Shifted to gear {currentGear}");
    }
    public void Accelerate()
    {
        if (!isEngineOn)
        {
            Console.WriteLine($"{brand} {model} : Engine is off! Cannot accelerate.");
            return;
        }
        currentSpeed += 10;
        Console.WriteLine($"{brand} {model} : Accelerating... Current speed: {currentSpeed} km/h");
    }
     public void Brake()
    {
        currentSpeed -= 20;
        if (currentSpeed < 0) currentSpeed = 0;
        Console.WriteLine($"{brand} {model} : Braking! Speed is now {currentSpeed} km/h");
    }
    public void StopEngine()
    {
        isEngineOn = false;
        currentGear = 0;
        currentSpeed = 0;
        Console.WriteLine($"{brand} {model} : Engine turned off.");
    }
}
class Encapsulation
{
    public static void Main(string[] args)
    {
        Car c=new Car("Toyota","Camry","Bridgestone");
        c.StartEngine();
        c.ShiftGear(1);
        c.Accelerate();
        c.Brake();
        c.StopEngine();
        Console.WriteLine("Current Speed: " + c.getCurrentSpeed());
    }
}
```